Old Thread life cycle :

* This is called old thread life cycle and It is available before JDK 1.5V

* A life cycle concept came from JDK 1.5V.

* As we know a thread is well known for **Indepnedent execution** but still its passes through differnt phases OR states.

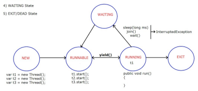
1) NEW State (Born State)

2) RUNNABLE State (Ready to run)

3) RUNNING State

4) WAITING State

5) EXIT/DEAD State



New State :-

Whenever we create a thread instance(Thread Object) a thread comes to new state OR born state. New state does not mean that the Thread has started yet only the object or instance of Thread has been created.

Runnable state :-

Whenever we call start() method on thread instance, A thread moves to Runnable state i.e Ready to run state. Here Thread schedular is responsible to select/pick a particular Thread from Runnable state and sending that particular thread to Running state for execution.

Running state :-

If a thread is in Running state that means the thread is executing its own run() method in a separate runtime stack Memory.

From Running state a thread can move to waiting state either by an order of thread schedular OR developer has written some method(wait() or join() or sleep()) to put the thread into temporarly waiting state.

From Running state the Thread may also move to Runnable state directly, if developer has written Thread.yield() method explicitly.

Waiting state :-

A thread is in waiting state means it is waiting for it's time period to complete OR in some cases it is also waiting for lock (monitor) OR another thread to complete. Once the time period will be completed then it will re-enter inside the Runnable state to complete its remaining task.

Dead or Exit :

Once a thread has successfully completed its run method in the corresponding stack then the thread will move to DEAD state. Please remember once a thread is dead we can't restart a thread in java.

IQ :- If we write Thread.sleep(1000) then exactly after 1 sec the Thread will re-start?

Ans :- No, We can't say that the Thread will directly move from waiting state to Running state.

The Thread will definetly wait for 1 sec in the waiting state and then again it will re-enter into Runnable state which is control by Thread Schedular so we can't say that the Thread will re-start just after 1 sec.

Anonymous inner class with reference variable using Thread class :

```
package com.ravi.anonymous;

public class AnonymousInnerClassWithReference
{
    public static void main(String[] args)
    {
        Thread t1 = new Thread()
        {
            @Override
            public void run()
            {
                String name = Thread.currentThread().getName();
                IO.println("Current thread name is :"+name);
            }
        };

        t1.start();
    }

}
```

Anonymous inner class without reference with Thread class :

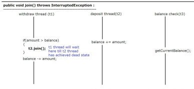
```
package com.ravi.anonymous;

public class AnonymousInnerClassWithoutReference
{
   public static void main(String[] args)
   {
       new Thread()
       {
         @Override
         public void run()
         {
             String name = Thread.currentThread().getName();
             IO.println("Thread Name is :"+name);
         }
       }.start();

   }
}
```

---

public void join() throws InterruptedException :



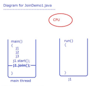
The main purpose of join() method to put the current thread into temporarly waiting state until the other thread finish its execution.

Here the currently executing thread stops its execution and the thread goes into the waiting state. The current thread remains in the wait state until the thread on which the join() method is invoked has achieved its dead state.

It also throws checked exception i.e InterruptedException so better to use try catch or declare the method as throws.

It is a non static method so we can call this method with the help of Thread object reference.

Diagram for JoinDemo1.java




```
package com.ravi.join;

class Join extends Thread
{
    @Override
    public void run()
    {
        String name = Thread.currentThread().getName();
        IO.println(name+" thread has started!!!");

        for(int i=1; i<=5; i++)
        {
           IO.println(i+" by "+name+" thread");
           try
           {
               Thread.sleep(1000);
           }
           catch(InterruptedException e)
           {

           }
        }

        IO.println(name+" thread has ended!!!");
    }
}

public class JoinDemo1
{
    public static void main(String[] args) throws InterruptedException
    {
        IO.println("Main thread started");

        Join j1 = new Join();
        Join j2 = new Join();
        Join j3 = new Join();

        j1.setName("J1"); j2.setName("J2"); j3.setName("J3");

        j1.start();

        j1.join(); //main thread will wait for j1 thread to complete

        IO.println("Main thread wake up");

        j2.start();
        j3.start();
        IO.println("Main thread ended");
    }

}
```

```
package com.ravi.join;

public class JoinDemo2
{
    public static void main(String[] args)
    {
        IO.println("Main thread started");
        Alpha a1 = new Alpha();
        a1.setName("Alpha");
        a1.start();
        IO.println("Main thread ended");
    }
}

class Alpha extends Thread
{
    @Override
    public void run()
    {
        Beta  beta = new Beta();
        beta.setName("Beta");
        beta.start();
        try
        {
            beta.join(); //Alpha thread will wait here for Beta thread to complete
        }
        catch(InterruptedException e)
        {

        }

        String name = Thread.currentThread().getName();
        IO.println(name+" thread re-started");

        for(int i=1; i<=10; i++)
        {
            IO.println(i+" by "+name+" thread");
        }


    }
}

class Beta extends Thread
{
   @Override
   public void run()
   {
       String name = Thread.currentThread().getName();

       for(int i=1; i<=20; i++)
       {
           IO.println(i+" by "+name+" thread");
           try
           {
               Thread.sleep(500);
           }
           catch(InterruptedException e)
           {
               e.printStackTrace();
           }
       }
       IO.println(name+" thread completed!!!");
   }
}
```

```
package com.ravi.join;
//Online banking amount transfer after OTP Verification

class OTPVerification extends Thread
{
    @Override
    public void run()
    {
        IO.println("OTP verificattion is in progress");
        try
        {
            Thread.sleep(2000);
        }
        catch(InterruptedException e)
        {

        }
        IO.println("OTP verificattion successful");
    }
}

class BalanceCheck extends Thread
{
    @Override
    public void run()
    {
        IO.println("Balance verificattion is in progress");
        try
        {
            Thread.sleep(1500);
        }
        catch(InterruptedException e)
        {

        }
        IO.println("Balance verificattion successful");
    }
}

public class JoinDemo4
{
    public static void main(String[] args) throws InterruptedException
    {
       OTPVerification otp = new OTPVerification();
       BalanceCheck bal = new BalanceCheck();

       IO.println("Initiating the balance transfer");
       otp.start();
       bal.start();

       bal.join();
       otp.join();

       IO.println("Amount transfer successfully");
    }
}
```